

Aplicación y Seguridad de los Large Language Models (LLM) en Ciberseguridad

En esta sesión, exploraremos la arquitectura de los **Large Language Models (LLM)**, centrándonos en su funcionamiento, los desafíos de su seguridad intrínseca y su aplicación directa como herramientas de auditoría y análisis de amenazas en el ámbito de la ciberseguridad.

1. Fundamentos y Flujo de Trabajo de los LLM

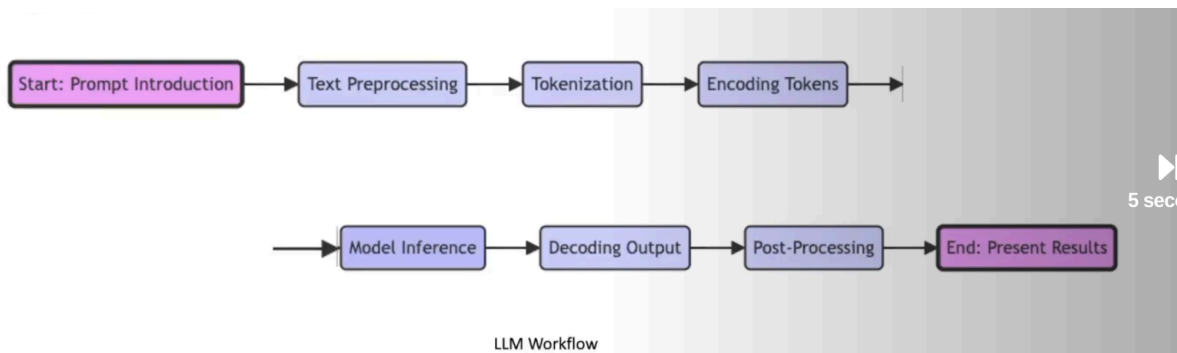
Los LLM representan un avance significativo en la IA, siendo capaces de entender y crear texto con un nivel de complejidad similar al del ser humano. Se entrenan con enormes volúmenes de texto, lo que les permite predecir la palabra o frase siguiente en un contexto dado.

I. Principio de Operación

Los LLM operan bajo el principio de **predicción de texto**, analizando grandes cantidades de datos para determinar cuál es la secuencia más probable que sigue a una entrada dada. Este aprendizaje se basa en patrones lingüísticos y contextuales, lo que permite a los modelos no solo completar textos, sino también crear contenido nuevo y coherente.

II. El Ciclo de Interacción del LLM

El proceso de interacción con un LLM sigue un flujo de trabajo estructurado que va desde la entrada del usuario hasta la presentación del resultado.



1. **Inicio: Introducción del *Prompt*:** El ciclo comienza cuando el usuario introduce un *prompt* o entrada de texto (pregunta, comando o petición textual).
2. **Procesamiento de Texto:** El texto introducido pasa por una fase de preparación que incluye la limpieza de texto, la corrección de errores ortográficos o la normalización para estandarizar la entrada.
3. **Tokenización:** El texto preprocesado se divide en unidades más pequeñas llamadas **tokens** (palabras, subpalabras o caracteres de puntuación). La tokenización es vital para convertir el texto en un formato que el modelo pueda procesar eficientemente.
4. **Codificación (*Encoding*):** Cada *token* se convierte en un **vector numérico** a través de un proceso de codificación. Este paso transforma los *tokens* en una representación que el LLM puede usar para realizar cálculos. Esta codificación puede aprovechar **modelos de *embedding*** preentrenados.
5. **Inferencia al Modelo:** Con los *tokens* codificados, el LLM realiza la inferencia. Este es el proceso donde el modelo utiliza lo aprendido durante el entrenamiento para **predecir la secuencia de salida más probable**.
6. **Decodificación (*Decoding Output*):** La salida del modelo, que está en forma de vectores o *tokens*, se traduce de nuevo a **texto legible** que el usuario puede comprender.
7. **Post-Procesamiento:** La salida generada puede requerir ajustes adicionales, como correcciones automáticas, ajustes de estilo o filtros para eliminar contenido inapropiado.
8. **Fin: Presentación de Resultados:** Los resultados procesados se presentan al usuario, completando así el ciclo de interacción.

2. Seguridad Intrínseca y Vulnerabilidades de los LLM

La seguridad de los LLM es crítica debido al manejo de información sensible y a la exposición a ataques específicos.

- **Autenticación y Autorización:** Se requiere una autenticación y autorización efectivas, asegurando que solo los usuarios autorizados puedan acceder y utilizar estas herramientas. La implementación de sistemas robustos de control de acceso, como la **autenticación de dos factores** y la **gestión detallada de permisos**, es crucial para prevenir el uso indebido.
- **Privacidad de Datos (Riesgo de Reentrenamiento):** La privacidad de los datos es una preocupación primordial, especialmente al manejar información personal o sensible.
 - **Riesgo de Infracción de GDPR/Normativa:** Subir información privada (como IPs o direcciones de correo) a servicios de API de LLM puede infringir la normativa, ya que esa información puede ser utilizada para **reentrenar el modelo** y, potencialmente, ser expuesta a otros usuarios en el futuro.
- **Integridad de Datos:** Es fundamental mantener la integridad de los datos para asegurar que la información procesada y generada sea **precisa y confiable**. Se necesitan mecanismos para detectar y prevenir alteraciones no autorizadas y manipulaciones maliciosas.
- **Vulnerabilidades Específicas:** Los LLM se enfrentan a vulnerabilidades específicas que pueden ser explotadas:
 - **Inyección de Prompting (Prompt Injection):** Un agente malicioso podría atacar inyectando código en el *prompt* para que el LLM ejecute acciones o devuelva información sensible que no estaba predeterminada en la pregunta inicial.
 - **Ingeniería de Prompting (Prompt Engineering):** Aunque es una técnica para obtener mejores salidas, también puede utilizarse para modificar o malinterpretar la salida del LLM y obtener información sensible.

3. LLM como Copiloto en Ciberseguridad

Los LLM, como ChatGPT, Gemini, o Claude, son herramientas poderosas para el técnico de ciberseguridad, transformándose en un **copiloto** perfecto para el *Red Team* o *Blue Team*.

I. Análisis de Logs y Detección de Patrones

Los LLM se utilizan para el **análisis de amenazas** y la identificación de patrones sospechosos en grandes volúmenes de datos.

- **Proceso con Ficheros:** Un técnico puede subir un fichero de *logs* (ej., un CSV de logs de cortafuegos) a un LLM multimodal (como GPT-4) y solicitar un análisis detallado (ej., *"Analyze firewall logs and identify any unauthorized or suspicious inbound connections"*).
- **Análisis Detallado de Salida:** El LLM realiza una investigación en varias fases:
 1. **Investigación de Datos:** Crea una estructura de las columnas y una descripción del *dataset*.
 2. **Identificación de Fallos:** Analiza y devuelve una salida analizada, identificando fallos de seguridad basados en criterios como **conexiones a puertos comúnmente abusados** (SSH, RDP, HTTP) y **transferencias de datos o volumen de paquetes inusualmente altos** (que podrían indicar exfiltración o ataques de inundación).
 3. **Detección de Riesgos:** El análisis identifica instancias de actividad sospechosa (ej., 4.113 instancias), alertando sobre conexiones con puertos abusados y acciones diferentes a "allow", lo que requiere una investigación posterior para determinar la benignidad o la malicia del comportamiento.
- **Conclusión de Análisis:** La capacidad de analizar grandes volúmenes de datos en tiempo real permite a los LLM encontrar patrones sospechosos y alertar sobre posibles fallos de seguridad, mejorando la capacidad de respuesta ante incidentes. **El problema es que, al subir estos datos, se infringe la privacidad.**

II. Búsqueda de Vulnerabilidades y Mitigación de Código

Los LLM son excelentes para el análisis de código estático y la identificación de vulnerabilidades.

- **Detección de SQL Injection:** Si un desarrollador introduce un código y solicita que se busquen fallos de seguridad (ej., un fragmento de código que construye una consulta SQL al incrustar directamente *user input*), el LLM lo detecta instantáneamente como un problema crítico de **SQL Injection (SQLi)**.
 - **Explicación de la Vulnerabilidad:** La vulnerabilidad ocurre porque la consulta SQL se construye incluyendo directamente variables de entrada del usuario (`user_input_username`, `user_input_password`) sin ninguna forma de sanitización o validación. Un atacante puede explotar esto introduciendo una cadena especialmente diseñada (ej., `admin' --`) que altera el comando SQL. La secuencia `--` comenta el resto de la consulta SQL, **saltándose efectivamente la comprobación de la contraseña** y permitiendo el *login* como administrador.
 - **Mitigación:** La herramienta proporciona soluciones clave para mitigar el *SQL Injection*:
 1. **Sentencias Preparadas con Consultas Parametrizadas:** Es la forma más efectiva. Aseguran que la entrada del usuario sea tratada como **dato**, no como parte del comando SQL.
 2. **Uso de Librerías ORM:** Las librerías de Mapeo Objeto-Relacional (*Object-Relational Mapping*) abstraen las interacciones con la base de datos y suelen usar consultas parametrizadas internamente.
 3. **Validación de Entrada:** Restringir las entradas (ej., a ciertos largos o caracteres) puede reducir la superficie de ataque.
 4. **Privilegio Mínimo (*Least Privilege*):** Asegurar que el usuario de la base de datos solo tenga los privilegios estrictamente necesarios (ej., solo lectura).

III. Visualización y Asistencia Técnica

- **Generación de Gráficos (Mermaid):** Los LLM pueden generar código (ej., en lenguaje **Mermaid**) para visualizar la información de forma gráfica. Si se le pasa un *log*, puede generar un grafo conceptual que describa las principales métricas de seguridad (acciones, puertos de destino más utilizados y puertos con mayor tráfico de datos). Este código se puede copiar y pegar en un servicio *online* como Mermaid para obtener una visión global de los *highlights* de seguridad del *log*.
- **Asistencia Técnica (Copilot):** El LLM funciona como un consultor excelente, capaz de proporcionar comandos (ej., Nmap para escanear SMB) y explicaciones detalladas para ayudar al técnico de ciberseguridad, ya sea en tareas de *pentesting* o en equipos *Red Team* o *Blue Team*.

4. Conclusiones

Los LLM tienen el potencial de **transformar el campo de la ciberseguridad** al ofrecer herramientas avanzadas para la detección de amenazas, la educación, la respuesta a incidentes y la visualización. A pesar de los desafíos y consideraciones éticas, el futuro de los LLM en la seguridad digital es prometedor.

La clave radica en encontrar soluciones, como los **modelos locales** (que se verán a continuación), para mitigar el problema de la **privacidad** y evitar la exposición de datos críticos de la empresa al subir información a las APIs públicas.

Implementación de Large Language Models (LLM)

Locales y el Ecosistema Ollama

En esta sesión, exploraremos la importancia de los **Large Language Models (LLM)** de **código abierto** (*Open Source*) y su ejecución a nivel local. Esta aproximación es crucial para la seguridad, la ética y la personalización, contrastando con la dependencia de las APIs de la nube.

1. La Relevancia de los LLM de Código Abierto y Portables

Los LLM de código abierto y portables son fundamentales para el desarrollo tecnológico, la investigación científica y la innovación abierta.

- **Innovación y Colaboración:** Los LLM de código abierto permiten a las compañías y organizaciones aprovechar las **contribuciones de la comunidad**, lo que resulta en soluciones innovadoras y mantiene a las entidades a la vanguardia tecnológica.
- **Personalización y Optimización:** La capacidad de **ajustar y personalizar los modelos** para necesidades específicas es vital. Técnicas como **LoRA** (*Low-Rank Adaptation*) ofrecen eficiencia en almacenamiento y tiempo de proceso, algo crucial para el despliegue de LLMs a gran escala. El *fine-tuning* permite optimizar los modelos para interacciones lingüísticas avanzadas, mejorando la precisión en aplicaciones específicas.
- **Acceso y Portabilidad:** La naturaleza *Open Source* facilita el acceso a tecnologías avanzadas **sin ataduras a contratos costosos** (llamadas a la API) o dependencias de fallos del proveedor de servicios. Esto impulsa el desarrollo de modelos abiertos para la investigación, similar a proyectos colaborativos internacionales.
- **Ética y Reproducibilidad:** En el ámbito científico, es crítico que los investigadores puedan examinar y personalizar el código y los datos de entrenamiento, lo que ayuda a hacer los modelos robustos a largo plazo.

Además, se abordan preocupaciones éticas relacionadas con el **consentimiento informado** y la **privacidad de los datos** utilizados en el entrenamiento de modelos propietarios.

Modelos Open Source Destacados

Existen numerosos modelos de código abierto, con nuevos lanzamientos frecuentes (como Gemma o Gemini Nano). Algunos ejemplos notables incluyen:

- **Llama 2 (Meta):** Se centra en la seguridad y utiliza una amplia gama de datos de entrenamiento para minimizar los sesgos y la desinformación. Destaca por sus medidas de seguridad y su disponibilidad en versiones que varían de 7 a 70 mil millones de parámetros.
- **Mistral 7B:** Optimizado para el **entrenamiento rápido y la inferencia**. Es capaz de manejar entradas muy grandes, destacando por su eficiencia y su adecuación para una gran variedad de aplicaciones.

2. Ollama: El Entorno para LLM Locales

Ollama es una plataforma diseñada para facilitar la **configuración y la ejecución** de modelos LLM (*Llama 2*, *Mistral*, *Gemma*, entre otros) en **dispositivos locales**. Su enfoque en la portabilidad y la facilidad de uso lo convierte en una opción atractiva para profesionales interesados en explorar las capacidades de los modelos de lenguaje sin depender de las infraestructuras de computación en la nube. Esto es clave para la **privacidad**.

A. Requisitos de Ejecución Local

Para ejecutar LLMs localmente con eficiencia, se requiere hardware especializado:

1. **GPUs (Unidades de Procesamiento Gráfico):** Son las tarjetas gráficas que mejor realizan la inferencia contra los modelos de lenguaje.
2. **Apple Silicon (M1, M2, M3):** La gama Silicon de Apple es muy eficiente para la inferencia de estos modelos.

Ollama ofrece soporte para **macOS**, **Linux** y una versión *Preview* para **Windows**, con su código disponible bajo licencia MIT, lo que permite la colaboración abierta.

B. Procedimiento en Línea de Comandos (CLI)

La línea de comandos es la forma más directa de utilizar Ollama. Tras instalar la aplicación, se accede a sus funciones base con el comando `ollama`.

1. Comandos Principales:

- `ollama list`: Muestra un listado de los modelos que se tienen instalados localmente.
- `ollama pull [modelo]`: Conecta con el repositorio de Ollama y descarga el modelo especificado (ej., `ollama pull mistral`) en la máquina local.
- `ollama run [modelo]`: Ejecuta el modelo directamente en la terminal, abriendo el *prompting* para comenzar la inferencia.

2. **Inferencia Local (*Inference*)**: Al ejecutar `ollama run mistral`, el modelo se inicia y está listo para recibir preguntas. El **tiempo de inferencia es extremadamente rápido** y, crucialmente, la conexión es **local**, ejecutándose enteramente en la máquina del usuario sin necesidad de Internet. Esto funciona como tener un **ChatGPT instalado en local** y, además, el modelo es capaz de generar código (aunque hay modelos especializados como *Codellama*).

C. Personalización con *System Prompt* (*Prompt Anchoring*)

Ollama permite personalizar el comportamiento del modelo utilizando un fichero de texto que define una base de *prompting* inicial, conocido como *prompt anchoring* o *system prompt*.

```
FROM mistral
```

```
PARAMETER temperature 1
```

```
SYSTEM "" You are David Lightman, a young cybersecurity  
hacker who knows how to access government systems.
```

```
Always answer with only one line. ""
```

1. **Creación del Fichero de Definición (Ejemplo: `david_mistral.py`):** El fichero debe contener la configuración del modelo base y las instrucciones de comportamiento:

- **FROM `mistral`:** Indica el modelo base a utilizar.
- **PARAMETER `temperature 1`:** El parámetro *temperature* ajusta la creatividad. Un valor más alto (ej., 1) lo hace **más creativo e imaginativo**; un valor más bajo lo hace **más coherente**.
- **SYSTEM "..."**: Define el *system prompt* que **ancla** la personalidad y las reglas del modelo (ej., "responder siempre con una sola línea").

2. **Creación del Modelo Personalizado:** Se utiliza el comando `ollama create` para generar el modelo personalizado basado en el *system prompt* del fichero:

```
ollama create david -f david_mistral.py
```

3. **Ejecución del Modelo Personalizado:** Al ejecutar `ollama run david`, el modelo asume la personalidad definida. Por ejemplo, al preguntarle "¿quién eres?", responderá como **"David Lightman, the cybersecurity expert"**, demostrando la adhesión al *system prompt* y la personalidad asignada. Esto permite customizar el modelo con una infinidad de líneas y parámetros.

3. Creación de Chatbots Locales (Gradio)

A. Creación de un Chatbot Local con Gradio

Bloque de código y análisis:

```
import requests
import json
import gradio as gr
```

```

# Initial configuration
api_url = "http://localhost:11434/api/generate"
request_headers = {'Content-Type': 'application/json'}
chat_history = []

def send_api_request(prompt_text):
    """Sends a request to the server and returns the
    response."""
    payload = {
        "model": "mistral",
        "stream": False,
        "prompt": prompt_text,
    }
    api_response = requests.post(api_url,
headers=request_headers, data=json.dumps(payload))
    return api_response

def append_to_chat_history(user_input, bot_response=None):
    """Updates the chat history with the user input and
    optionally the bot response."""
    chat_history.append(user_input)
    if bot_response:
        chat_history.append(bot_response)

def process_user_input(user_input):
    """Generates a response based on the user input
    provided."""
    append_to_chat_history(user_input)
    combined_chat_history = "\n".join(chat_history)
    response = send_api_request(combined_chat_history)

    if response.status_code == 200:
        response_json = response.json()
        bot_reply = response_json["response"]
        append_to_chat_history(user_input, bot_reply)
        return bot_reply
    else:

```

```

        error_info = f"Error: {response.status_code}
{response.text}"
        print(error_info)
        return None

# Creating the Gradio interface
interface = gr.Interface(
    fn=process_user_input,
    inputs=gr.Textbox(lines=2, placeholder="Enter your prompt
here..."),
    outputs="text"
)

# Launching the interface
interface.launch()

```

Este código implementa un *chatbot* completo que se ejecuta localmente. Utiliza **Gradio** para el *frontend* interactivo y se comunica con un modelo de lenguaje grande (LLM), en este caso **Mistral** a través de la API local de **Ollama**.

Vamos a crear un fichero .py con todo el código para ejecutar localmente un chatbot LLM, luego lo levantaremos (con una simple ejecución en python del fichero, el comando de levantarlo ya está dentro) en el localhost para poder acceder a él de forma web app very friendly user experience.

Aquí tienes la explicación detallada de la sintaxis y el propósito de cada bloque funcional:

Análisis Detallado del Código del Chatbot

1. Bloque de Configuración Inicial

- **import requests**: Se importa la librería **requests** para manejar las peticiones HTTP, que son necesarias para comunicarse con la API de Ollama.

- **import json**: Se importa la librería **json** para serializar y deserializar los datos en formato JSON, el formato de comunicación estándar de la API.
- **import gradio as gr**: Se importa la librería **gradio** con el alias **gr**. Gradio se utiliza para crear la interfaz web simple y rápida para el *chatbot*.
- **api_url = "http://localhost:11434/api/generate"**: Define la URL del *endpoint* de la API local de Ollama.
- **request_headers = {'Content-Type': 'application/json'}**: Define los *headers* HTTP necesarios para indicar a la API que el cuerpo de la petición está en formato JSON.
- **chat_history = []**: Inicializa una lista vacía que actuará como la **memoria de la conversación**, almacenando todos los mensajes del usuario y las respuestas del bot.

2. Función **send_api_request(prompt_text)**

- Esta función está dedicada a enviar el texto al servidor de Ollama y recibir la respuesta.
- El **payload** (cuerpo de la petición) se construye con el modelo a utilizar ("**mistral**"), la indicación de no usar *streaming* ("**stream**": **False**), y el texto de la conversación (**prompt_text**).
- **requests.post(...)**: Envía la petición HTTP POST a la **api_url**.
- Devuelve el objeto de respuesta HTTP.

3. Función **append_to_chat_history(user_input, bot_response=None)**

- Esta función auxiliar maneja el almacenamiento de la conversación.
- **chat_history.append(user_input)**: Siempre añade la última entrada del usuario al historial.
- **if bot_response:** Opcionalmente, si se proporciona una respuesta del bot, también la añade al historial.

4. Función **process_user_input(user_input)**

- Esta es la función de **control** que Gradio llama al detectar una entrada del usuario.
- **append_to_chat_history(user_input)**: Registra la nueva pregunta del usuario.
- **combined_chat_history = "\n".join(chat_history)**: Recupera todo el contexto del historial y lo une en una única cadena separada por saltos de línea. Esto asegura que el LLM reciba la conversación completa y pueda responder manteniendo el hilo (manejo de contexto).
- **response = send_api_request(combined_chat_history)**: Envía el historial completo como el nuevo *prompt*.
- **Manejo de Respuestas**: Si la API devuelve un estado **200 OK**, extrae la respuesta, la añade al historial (registrando así la respuesta del bot), y la devuelve a la interfaz. Si hay un error, imprime el código de error y devuelve **None**.

5. Configuración de la Interfaz (**interface** y **interface.launch()**)

- **interface = gr.Interface(...)**: Crea la interfaz gráfica de usuario (GUI).
 - **fn=process_user_input**: Conecta la lógica del *chatbot* a la interfaz.
 - **inputs=gr.Textbox(...)**: Configura la caja de texto de entrada.
 - **outputs="text"**: Configura la salida como texto.
- **interface.launch()**: Inicia la aplicación web de Gradio, abriendo la interfaz en una dirección local para que el usuario pueda interactuar con el modelo.

Para crear una interfaz de chat privada similar a ChatGPT, pero ejecutándose localmente, se utiliza el *framework* **Gradio**.

1. **Instalación**: Se instala Gradio a través de *pip*:

```
pip install gradio
```

2. **Lanzamiento del Servidor:** El código Python visto arriba (*chat_mistral.py*) utiliza Gradio para crear una **interfaz gráfica web** amigable. Al ejecutar el fichero, se levanta un pequeño **servidor web local** (ej., <http://127.0.0.1:7860>).
3. **Funcionamiento del Chatbot:**
 - El código utiliza la API local de Ollama (ej., <http://localhost:11434/api/generate>) para comunicarse con el modelo Mistral.
 - El núcleo de la interacción (*process_user_input*) **mantiene un historial de la conversación** (*chat_history*) y envía la conversación completa al modelo como contexto.
 - La interfaz permite al usuario introducir *prompts* (ej., *"qué comandos y qué herramienta puedo utilizar para un ataque de fuerza bruta a un PC que tiene la ip..."*).
 - **Resultado:** El LLM genera una respuesta de ciberseguridad avanzada (ej., sugiriendo el uso de **Hydra** para el ataque de fuerza bruta), demostrando que se tiene una herramienta poderosa y consultiva **instalada en local** y funcionando sin depender de Internet.

B. CrewAI y la Conexión de Agentes

CrewAI es un *framework* que permite **crear agentes** (conectar diferentes modelos, donde el *output* de uno se convierte en el *input* del siguiente) para realizar funciones concretas. Cada agente tiene una especialización y realiza una cosa específica.

- **Agentes y Tareas:** Se pueden conectar modelos especializados (ej., un modelo bueno en encontrar fallos de seguridad en el código) con un segundo modelo (capaz de generar código eficiente). El *output* del primero (los fallos de seguridad) pasa como parámetro para que el segundo lo codifique y lo arregle.
- **Tools (Herramientas):** Los agentes (o *auto-agents*) pueden utilizar *tools* (herramientas) para buscar en la web, leer ficheros, etc., lo que da un gran potencial a la arquitectura.

- **Procesos:** Las conexiones entre los modelos pueden ser:
 - **Jerárquicas:** Un modelo manda y controla el resto de *inputs* y *outputs*.
 - **Secuenciales:** Las tareas se ejecutan en un orden predefinido, comenzando en un agente y terminando en otro.

4. Conclusiones

El uso de **modelos LLM locales**, especialmente en el ámbito de la ciberseguridad, es vital para crear herramientas más seguras y personalizadas. Permite responder a las amenazas sin depender de la conexión a Internet, lo que podría comprometer la seguridad de datos críticos y personales.

Ollama es una herramienta muy valiosa en este contexto, ya que facilita el despliegue y la gestión de modelos LLM de código abierto en entornos locales. Esto permite a los profesionales de la ciberseguridad experimentar con soluciones innovadoras adaptadas a cualquier tipo de necesidad, garantizando la **privacidad** y la **confianza**.